Running **Valgrind** is a great way to catch memory leaks, invalid memory access, and other sneaky bugs in your C or C++ programs. Here's a quick guide to get you started:

Step-by-Step: Running Valgrind

1. **Compile your program with debugging info**
   - This allows Valgrind to show you line numbers and variable names.

   ```
   gcc -g your_program.c -o your_program
   ```

2. **Run your program with Valgrind**
   - Use the `--leak-check=full` option for detailed memory leak info.

   ```
   valgrind --leak-check=full ./your_program
   ```

3. **Interpret the output**
   - Valgrind will show:
     - Invalid reads/writes
     - Use of uninitialized memory
     - Memory leaks (with stack traces)
     - Mismatched `malloc`/`free` calls

Example

```
#include <stdlib.h>

int main() {
    int *arr = malloc(10 * sizeof(int));
    arr[10] = 42;  // Oops! Out-of-bounds write
    return 0;
}
```

Compile and run:

```
gcc -g test.c -o test
valgrind --leak-check=full ./test
```

Valgrind will catch the out-of-bounds write and report it.

Useful Options

Option	Description
`--leak-check=full`	Detailed info on memory leaks
`--track-origins=yes`	Shows where uninitialized values come from
`--show-leak-kinds=all`	Displays all types of leaks

Option	Description
`--log-file=valgrind.log`	Saves output to a file

Want help interpreting Valgrind output or debugging a specific error? Just drop your code snippet and I’ll walk you through it.